HW3_Report

Implementation Steps

> ✎ همان‌طور که در حل تمرین گفته شد، در طول تمرین سعی کردم این‌ها را در قالب **function** و جامع پیاده سازی کنم تا بعدا هم از آن‌ها استفاده کنیم

ADX

برای پیاده‌سازی این بخش، خود داک یک نقشه راه بسیار خوبی داشت و بنابراین می‌توانیم از آن به عنوان یک پایه خوب استفاده کنیم

1. Directional Moving (DM)

فرمول‌های این بخش به شکل زیر هستند:

$$+DM_t = H_t - H_{t-1}$$
$$-DM_t = L_{t-1} - L_t$$

و می‌توانیم آن را به این شکل پیاده‌سازی کنیم:

```
df['H_diff'] = df['High'] - df['High'].shift(1)
df['L_diff'] = df['Low'].shift(1) - df['Low']
```

و حالا برای تاریخ‌ها محاسبه می‌کنیم که آیا `DMt+` بزرگ‌تر است یا `DMt-` و مقدار بزرگ‌تر را برای آن انتخاب می‌کنیم

```
for i in range(1, len(df)):
    h_diff = df['H_diff'].iloc[i]
    l_diff = df['L_diff'].iloc[i]

    if h_diff > l_diff and h_diff > 0:
        df.loc[df.index[i], '+DM'] = h_diff
    elif l_diff > h_diff and l_diff > 0:
        df.loc[df.index[i], '-DM'] = l_diff
```

2. True Range (TR)

فرمول آن به این شکل است:

$$TR_t = max(H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|)$$

در کد می‌توانیم به سادگی سه term را محاسبه کنیم و تابع max را برای به دست آوردن نتیجه استفاده کنیم:

```
df['TR1'] = df['High'] - df['Low']
df['TR2'] = abs(df['High'] - df['Close'].shift(1))
```

```
df['TR3'] = abs(df['Low'] - df['Close'].shift(1))
df['TR'] = df[['TR1', 'TR2', 'TR3']].max(axis=1)
```

دلیل استفاده از shift(1) این است که می‌خواهیم قیمت بسته شدن فعلی را با قیمت بسته شدن قبلی مقایسه کنیم، بنابراین باید ستون close را یک ردیف جابجا کنیم.
و `max(axis=1)` برای گرفتن بیشترین مقدار از سه ستون برای هر ردیف استفاده می‌شود.

3. Average True Range (ATR)

فرمول آن به این شکل است:

$$ATR_t = \frac{1}{n} \sum_{i=t-n+1}^{t} TR_i$$

با توجه به اینکه در document مقدار `n=14` داده شده است، می‌توانیم به سادگی از تابع rolling mean برای محاسبه ATR استفاده کنیم:

```
df['ATR'] = df['TR'].rolling(window=period).mean()
```

4. SMA DM

برای این بخش از یک میانگین متحرک ساده روی `DM` ها محاسبه می‌کنیم و می‌توانیم به سادگی آن را به این شکل بنویسیم:

$$SMA + DM = \frac{1}{n} \sum_{i=t-n+1}^{t} +DM_i$$

$$SMA - DM = \frac{1}{n} \sum_{i=t-n+1}^{t} -DM_i$$

و می‌توانیم دوباره از متد rolling برای به دست آوردن موارد بالا استفاده کنیم

```
df['SMA_+DM'] = df['+DM'].rolling(window=period).mean()
df['SMA_-DM'] = df['-DM'].rolling(window=period).mean()
```

> دوره برای این بخش نیز در document به صورت `n=14` داده شده است

5. Directional Indicators

در این بخش `DI+` و `DI-` را با استفاده از فرمول‌های زیر محاسبه می‌کنیم:

$$+DI_t = (\frac{SMA + DM_t}{ATR_t}) \times 100$$

$$-DI_t = (\frac{SMA - DM_t}{ATR_t}) \times 100$$

و می‌توانیم به سادگی آن را با استفاده از کد زیر محاسبه کنیم:

```
df['+DI'] = (df['SMA_+DM'] / df['ATR']) * 100
df['-DI'] = (df['SMA_-DM'] / df['ATR']) * 100
```

6. Directional Index

این بخش به بخش قبلی وابسته است و می‌توانیم آن را با استفاده از فرمول زیر محاسبه کنیم:

$$DX_t = \frac{|(+DI_t) - (-DI_t)|}{|(+DI_t) + (-DI_t)|} \times 100$$

و می‌توانیم به این شکل پیاده‌سازی کنیم:

```
di_diff = abs(df['+DI'] - df['-DI'])
di_sum = df['+DI'] + df['-DI']

df['DX'] = np.where(di_sum != 0, (di_diff / di_sum) * 100, 0)
```

7. Average Directional Index (ADX)

حال می‌توانیم شاخص ADX را بدست آوریم.
فرمول آن به این شکل است:

$$ADX_t = \frac{1}{n} \sum_{i=t-n+1}^{t} DX_i$$

و دوباره از متد rolling برای گرفتن میانگین DX استفاده می کنیم:

```
df['ADX'] = df['DX'].rolling(window=period).mean()
```

8. market state

و حالا که ADX را داریم می‌توانیم از آن برای تعیین وضعیت بازار استفاده کنیم.
اگر شاخص بیشتر از 25 باشد (که در document داده شده است) بازار trending دارد، در غیر این صورت بازار sideway هست.

کد نهایی کامل به این شکل است:

```
def calculate_adx(df, period=ADX_Period, adx_threshold=ADX_Threshold):

    df['H_diff'] = df['High'] - df['High'].shift(1)
    df['L_diff'] = df['Low'].shift(1) - df['Low']

    df['+DM'] = 0.0
    df['-DM'] = 0.0

    for i in range(1, len(df)):
```

```
        h_diff = df['H_diff'].iloc[i]
        l_diff = df['L_diff'].iloc[i]

        if h_diff > l_diff and h_diff > 0:
            df.loc[df.index[i], '+DM'] = h_diff
        elif l_diff > h_diff and l_diff > 0:
            df.loc[df.index[i], '-DM'] = l_diff

    # TR
    df['TR1'] = df['High'] - df['Low']
    df['TR2'] = abs(df['High'] - df['Close'].shift(1))
    df['TR3'] = abs(df['Low'] - df['Close'].shift(1))
    df['TR'] = df[['TR1', 'TR2', 'TR3']].max(axis=1)

    # ATR
    df['ATR'] = df['TR'].rolling(window=period).mean()

    # DM sma
    df['SMA_+DM'] = df['+DM'].rolling(window=period).mean()
    df['SMA_-DM'] = df['-DM'].rolling(window=period).mean()

    # DI
    df['+DI'] = (df['SMA_+DM'] / df['ATR']) * 100
    df['-DI'] = (df['SMA_-DM'] / df['ATR']) * 100

    # DX
    di_diff = abs(df['+DI'] - df['-DI'])
    di_sum = df['+DI'] + df['-DI']

    df['DX'] = np.where(di_sum != 0, (di_diff / di_sum) * 100, 0)

    # ADX
    df['ADX'] = df['DX'].rolling(window=period).mean()

    df['market_state'] = np.where(df['ADX'] > adx_threshold, 'trending',
'sideways')

    return df
```

SMA Crossover Strategy

در این استراتژی یک میانگین متحرک کوتاه و بلند داریم، وقتی که sma کوتاه بالای sma بلند باشد، یک موقعیت long می‌گیریم، و وقتی پایین باشد، یک موقعیت short می‌گیریم.

برای پیاده‌سازی می‌توانیم دو sma با استفاده از متد rolling داشته باشیم:

```
def calculate_sma(df, short_window=Short_Window, long_window=Long_Window):

    df['SMA_short'] = df['Close'].rolling(window=short_window).mean()
```

```
    df['SMA_long'] = df['Close'].rolling(window=long_window).mean()

    df['SMA_diff'] = df['SMA_short'] - df['SMA_long']

    return df
```

Generating SMA Signals

> در این بخش همان‌طور که در doc گفته شد از crossover استفاده نمی‌کنیم، فقط چک می‌کنیم که آیا sma کوتاه بالای یا پایین sma بلند است و سیگنال را بر اساس آن انتخاب می‌کنیم. (نه crossover بین دو تا)

```
def get_sma_signals(df):
    sma_sig = pd.Series(0, index=df.index)
    sma_sig[df['SMA_short'] > df['SMA_long']] = 1

    sma_sig[df['SMA_short'] < df['SMA_long']] = -1

    return sma_sig.tolist()
```

منطق به این شکل کار می‌کند:

1. وقتی SMA کوتاه بالای SMA بلند است -> سیگنال = 1 (موقعیت long)
2. وقتی SMA کوتاه پایین SMA بلند است -> سیگنال = -1 (موقعیت short)
3. سیگنال به طور مداوم بر اساس موقعیت نسبی دو SMA حفظ می‌شود.

RSI Mean Reversion Strategy

برای پیاده‌سازی این استراتژی مراحل زیر را دنبال می‌کنیم:

1. Caculating The Price Change

برای این کار ابتدا باید ترم زیر را محاسبه کنیم:

$$\Delta_t = C_t - C_{t-1}$$

و در کد می‌توانیم به این شکل پیاده سازی کنیم:

```
df['price_change'] = df['Close'] - df['Close'].shift(1)
```

2. Calculating the Gain and Loss

فرمول این بخش به این شکل است:

$$G_t = max(\Delta_t, 0)$$

$$L_t = max(-\Delta_t, 0)$$

که می‌توان با استفاده از متد `where` و چک کردن مثبت یا منفی بودن تغییر قیمت انجام داد:

```
df['gain'] = np.where(df['price_change'] > 0, df['price_change'], 0)
df['loss'] = np.where(df['price_change'] < 0, abs(df['price_change']), 0)
```

3. Calculating the Average of the above

برای این بخش باید میانگینی از gain و loss که در مرحله 2 محاسبه کردیم بگیریم.

اگر بخواهیم فرمول آن را بنویسیم، به این شکل درمی‌اید:

$$\bar{G}_t = \frac{1}{n} \sum_{i=t-n+1}^{t} G_i$$

$$\bar{L}_t = \frac{1}{n} \sum_{i=t-n+1}^{t} L_i$$

و دوباره در کد از rolling و mean که تا به حال استفاده کرده‌ایم استفاده می‌کنیم:

```
df['avg_gain'] = df['gain'].rolling(window=period).mean()
df['avg_loss'] = df['loss'].rolling(window=period).mean()
```

4. Relative Strength

RS به سادگی میانگین gain تقسیم بر میانگین loss است:

$$RS_t = \frac{\bar{G}_t}{\bar{L}_t}$$

که یک تقسیم ساده از میانگین‌هایی است که در مرحله قبل به دست آوردیم:

```
df['RS'] = df['avg_gain'] / df['avg_loss']
```

5. Relative Strength Index (RSI)

بالاخره می‌توانیم rsi را محاسبه کنیم. برای rsi با استفاده از نتیجه rs می‌توانیم از فرمول زیر استفاده کنیم:

$$RSI_t = 100 - \frac{100}{1 + RS_t}$$

و در کد ما:

```
df['RSI'] = 100 - (100 / (1 + df['RS']))
```

این شکلی می‌شود.

و در آخر کد نهایی ما برای این بخش به این شکل درمی‌آید:

```
def calculate_rsi(df, period=RSI_Period):
    df['price_change'] = df['Close'] - df['Close'].shift(1)

    df['gain'] = np.where(df['price_change'] > 0, df['price_change'], 0)
    df['loss'] = np.where(df['price_change'] < 0, abs(df['price_change']),
0)

    df['avg_gain'] = df['gain'].rolling(window=period).mean()
    df['avg_loss'] = df['loss'].rolling(window=period).mean()

    df['RS'] = df['avg_gain'] / df['avg_loss']

    df['RSI'] = 100 - (100 / (1 + df['RS']))

    return df
```

Getting the signals from RSI

نحوه استفاده از RSI :

> ابتدا **RSI** را از داده‌های قبلی به دست می‌آوریم. سپس بررسی می‌کنیم که آیا **RSI** فعلی زیر آستانه‌ی `oversold` (که در داک مقدار ۳۰ دارد) قرار دارد؛ در این صورت یک موقعیت **long** می‌گیریم (`signal = 1`).
> همچنین اگر **RSI** فعلی بالای آستانه‌ی `overbought` باشد، یک موقعیت **short** می‌گیریم (`signal = -1`).

پس در کد هم ما دقیقا همین کار را انجام می‌دهیم:

```
def get_rsi_mean_reversion_signals(df, oversold=RSI_Oversold ,
overbought=RSI_Overbought):
    rsi = df['RSI']
    signals = [0] * len(df)
    for i in range(len(df)):
        if pd.notna(rsi.iloc[i]):
            if rsi.iloc[i] < oversold:
                signals[i] = 1
            elif rsi.iloc[i] > overbought:
                signals[i] = -1
            else:
                signals[i] = 0
    return signals
```

برای تمام روزها چک می‌کنیم که آیا rsi زیر یا بالای آستانه‌ها است و سیگنال‌ها را بر اساس آن تخصیص می‌دهیم. سیگنال‌ها عددی هستند: 1 برای پوزیشن لانگ، -1 برای پوزیشن شورت، و 0 برای هیچ سیگنال یا همان hold.

Combining Strategies

حالا که هر دو استراتژی SMA crossover برای بازارهای روندار و استراتژی RSI mean reversion برای بازارهای رنج را داریم، باید آن‌ها را بر اساس تعیین وضعیت بازار توسط شاخص ADX ترکیب کنیم.

- اگر ADX نشان می‌دهد که **بازار روندار است** (`ADX > 25`)، از **سیگنال‌های SMA crossover** استفاده می‌کنیم
- اگر ADX نشان می‌دهد که **بازار رنج است** (`ADX ≤ 25`)، از **سیگنال‌های RSI mean reversion** استفاده می‌کنیم

که به صورت زیر پیاده سازی می‌شود:

```
def apply_strategy_on_df(df):
    calculate_adx(df)
    calculate_sma(df)
    calculate_rsi(df)

    sma_signals = get_sma_signals(df)
    rsi_signals = get_rsi_mean_reversion_signals(df)

    final_signals = []
    for i in range(len(df)):
        market_state = df['market_state'].iloc[i]

        if market_state == 'trending':
            final_signals.append(sma_signals[i])
        elif market_state == 'sideways':
            final_signals.append(rsi_signals[i])
        else:
            final_signals.append(0)

    df['signal'] = final_signals

    return df
```

این توابع همه شاخص‌ها را محاسبه می‌کند، سیگنال‌های هر دو استراتژی را تولید می‌کند، و سپس سیگنال مناسب را بر اساس وضعیت فعلی بازار برای هر روز انتخاب می‌کند.

Backtesting

بعد از تولید سیگنال‌ها، باید استراتژی را بک‌تست کنیم تا عملکرد آن را ارزیابی کنیم. فرآیند backtesting شامل:

1. محاسبه درصد تغییر قیمت (با استفاده از Close یا Open بر اساس پارامتر `use_open_price`)
2. جابجایی سیگنال‌ها یک روز برای جلوگیری از look-ahead bias (ما بر اساس سیگنال امروز در open/close فردا عمل می‌کنیم)
3. **ضرب موقعیت در بازده برای به دست آوردن بازده خاص استراتژی
4. محاسبه بازده‌های تجمعی و ضرب در سرمایه اولیه

```
def backtest_strategy(df):
    price_col = 'Open' if use_open_price else 'Close'
    df['returns'] = df[price_col].pct_change().fillna(0)

    df['position'] = df['signal'].shift(1).fillna(0)
```

```
    df['strategy_returns'] = df['position'] * df['returns']

    df['cumulative_returns'] = (1 +
df['strategy_returns'].fillna(0)).cumprod()
    df['equity'] = df['cumulative_returns'] * initial_capital

    return df
```

Performance Metrics

برای ارزیابی استراتژی خود، چندین معیار عملکرد کلیدی را محاسبه می‌کنیم:

ابتدا این سه مقدار که در ادامه استفاده می‌شود را بدست میاریم:

```
equity = df['equity']
ret = df['strategy_returns'].dropna()
days = len(ret)
```

Return Metrics

1. **Net Profit**:

   مقدار مطلق دلاری که از سرمایه‌گذاری اولیه به دست آمده یا از دست رفته است.

   ```
   net_profit = equity.iloc[-1] - initial_capital
   ```

   توضیح: آخرین مقدار equity (سرمایه نهایی) را از سرمایه اولیه کم می‌کنیم تا سود یا زیان خالص به دلار را به دست آوریم. `iloc[-1]` آخرین عنصر سری را برمی‌گرداند.

2. **Total Return**:

   درصد سود یا زیان سرمایه‌گذاری اولیه در کل دوره.

   ```
   total_return = (equity.iloc[-1] / initial_capital) - 1
   ```

   توضیح: سرمایه نهایی را بر سرمایه اولیه تقسیم و سپس 1 کم می‌کنیم. نتیجه درصد تغییر کل است (مثلاً 0.5 یعنی 50% سود).

3. **CAGR (Compound Annual Growth Rate)**:

   نرخ بازدهی سالانه هموارشده، برای مقایسه عملکرد در بازه‌های زمانی مختلف مفید است.

$$CAGR = (1 + \text{Total Return})^{\frac{365}{\text{days}}} - 1$$

```
cagr = (1 + total_return) ** (Day_Per_Year / days) - 1 if days > 0 else 0
```

توضیح: بازده کل را به نرخ سالانه تبدیل می‌کنیم. ابتدا 1 را به total return اضافه می‌کنیم (برای فاکتور رشد)، سپس به توان `days/365` می‌رسانیم (برای سالانه‌سازی)، و در نهایت 1 کم می‌کنیم. شرط `if days > 0` از تقسیم بر صفر جلوگیری می‌کند.

5. **Gross Profit**:

مجموع تمام معاملات/دوره‌های برنده، که کل سودها را قبل از احتساب زیان‌ها نشان می‌دهد.

```
gross_profit = equity.diff().clip(lower=0).sum()
```

توضیح: `diff()` تفاوت روزانه equity را محاسبه می‌کند. `clip(lower=0)` تمام مقادیر منفی (زیان‌ها) را به صفر تبدیل می‌کند و فقط سودها باقی می‌مانند. `sum()` همه این سودهای روزانه را جمع می‌کند.

Risk Metrics

1. **Volatility**:

تغییرپذیری بازدهی را اندازه‌گیری می‌کند - مقادیر بالاتر نشان‌دهنده نوسانات قیمتی غیرقابل پیش‌بینی‌تر است.

```
volatility = ret.std() * np.sqrt(Day_Per_Year)
```

توضیح: ابتدا انحراف معیار بازده‌های روزانه را محاسبه می‌کنیم (`ret.std()`)، سپس در جذر تعداد روزهای سال ضرب می‌کنیم تا نوسانات سالانه به دست آید. این فرمول استاندارد برای سالانه‌سازی volatility است.

2. **Maximum Drawdown**:

بزرگترین افت درصدی از یک قله تا یک دره، که نشان‌دهنده بدترین زیانی است که یک سرمایه‌گذار تجربه می‌کرد.

```
roll_max = equity.cummax()
drawdown = (equity - roll_max) / roll_max
max_dd = drawdown.min()
```

توضیح: ابتدا حداکثر تجمعی سرمایه تا هر نقطه را محاسبه می‌کنیم (`cummax()`)، سپس تفاوت equity فعلی با اوج قبلی را بر اوج قبلی تقسیم می‌کنیم تا درصد افت را بدست آوریم. کمترین مقدار (`min()`) همان بدترین افت است.

3. **Maximum Drawdown Duration**:

طولانی‌ترین بازه زمانی که استراتژی برای بازگشت به اوج قبلی نیاز داشت، بر حسب روز.

```
is_dd = drawdown < 0
dd_duration = is_dd.groupby((~is_dd).cumsum()).cumsum().max()
```

توضیح: ابتدا روزهایی که در drawdown هستیم را مشخص می‌کنیم (`drawdown < 0`). سپس با `groupby` دوره‌های مختلف drawdown را جدا می‌کنیم و با `cumsum()` تعداد روزهای متوالی را می‌شماریم. `max()` طولانی‌ترین دوره را برمی‌گرداند.

Risk-Adjusted Return Metrics

1. **Sharpe Ratio**:

بازدهی به ازای هر واحد ریسک کل را اندازه‌گیری می‌کند - هرچه بالاتر، بهتر. مقادیر بالای 1 خوب و بالای 2 عالی است.

$$\text{Sharpe} = \frac{\text{Annualized Excess Return}}{\text{Volatility}}$$

```
rf_daily = (1 + risk_free_rate) ** (1 / Day_Per_Year) - 1
excess_ret = ret - rf_daily
ann_excess = excess_ret.mean() * Day_Per_Year
sharpe = ann_excess / volatility if volatility != 0 else 0
```

توضیح: ابتدا نرخ بدون ریسک سالانه را به روزانه تبدیل می‌کنیم، سپس بازده اضافی (`excess_ret`) را با کم کردن نرخ بدون ریسک از بازده استراتژی محاسبه می‌کنیم. میانگین بازده اضافی روزانه را در تعداد روزهای سال ضرب می‌کنیم تا سالانه شود، و در نهایت بر volatility (که قبلاً سالانه شده) تقسیم می‌کنیم. شرط `if volatility != 0` از تقسیم بر صفر جلوگیری می‌کند.

دلیل ضرب نکردن `sqrt(Day_Per_Year)` در کل نسبت

چون صورت و مخرج را *جداگانه* سالانه می‌کنیم. صورت (`ann_excess`) میانگین روزانه را در `Day_Per_Year` ضرب می‌کند و مخرج (`volatility`) انحراف معیار روزانه را در `sqrt(Day_Per_Year)` ضرب می‌کند. این به ما Sharpe سالانه را مستقیماً می‌دهد. فرم جایگزین عبارت است از: `* (daily_mean / daily_std)` که از نظر ریاضی معادل است اما به روش متفاوتی محاسبه می‌شود. `sqrt(Day_Per_Year)`

نکته درباره سالانه‌سازی: ابتدا میانگین بازده اضافی را سالانه می‌کنیم (میانگین روزانه را در `Day_Per_Year` ضرب می‌کنیم) و volatility را با ضرب انحراف معیار روزانه در `sqrt(Day_Per_Year)` سالانه می‌کنیم.
چون هر دو صورت و مخرج به شکل سالانه تبدیل می‌شوند، ضریب اضافی `sqrt(Day_Per_Year)` در نسبت نهایی لازم نیست. از نظر ریاضی دو فرم رایج معادل هستند:

- فرم سالانه (استفاده‌شده در بالا):

$$\text{Sharpe}_{ann} = \frac{\mu_{daily} \times D}{\sigma_{daily} \times \sqrt{D}}$$

- فرم استاندارد (Sharpe روزانه مقیاس‌شده):

$$\text{Sharpe}_{ann} = \left(\frac{\mu_{daily}}{\sigma_{daily}}\right) \times \sqrt{D}$$

هر دو Sharpe سالانه یکسانی تولید می‌کنند؛ در کد ما روش اول را پیاده‌سازی کردیم (صورت و مخرج را جداگانه سالانه می‌کنیم)، بنابراین `sqrt(Day_Per_Year)` اضافی لازم نیست.

2. **Sortino Ratio**:

مشابه Sharpe اما فقط volatility نزولی را جریمه می‌کند، نه سودهای صعودی - برای استراتژی‌های نامتقارن بهتر است.

$$\text{Sortino} = \frac{\text{Annualized Excess Return}}{\text{Downside Risk}}$$

```
downside_risk = np.sqrt((ret[ret < 0] ** 2).mean()) *
np.sqrt(Day_Per_Year)
sortino = ann_excess / downside_risk if downside_risk != 0 else 0
```

توضیح: برای محاسبه ریسک نزولی، ابتدا فقط بازده‌های منفی را انتخاب می‌کنیم (`ret[ret < 0]`)، آن‌ها را به توان 2 می‌رسانیم، میانگین می‌گیریم و جذر می‌کنیم تا انحراف معیار بازده‌های منفی بدست آید. سپس در `sqrt(Day_Per_Year)` ضرب می‌کنیم برای سالانه‌سازی. در نهایت بازده اضافی سالانه را بر این ریسک نزولی تقسیم می‌کنیم.

3. **Information Ratio**:

ثبات عملکرد بهتر نسبت به benchmark (S&P 500) را اندازه‌گیری می‌کند - مهارت در تولید alpha را نشان می‌دهد.

$$\text{Information Ratio} = \frac{\text{Annualized Active Return}}{\text{Tracking Error}}$$

```
spx_ret = spx['Close'].pct_change().reindex(ret.index).dropna()
active_ret = (ret - spx_ret).dropna()
tracking_err = active_ret.std() * np.sqrt(Day_Per_Year)
info_ratio = (active_ret.mean() * Day_Per_Year) / tracking_err if
tracking_err != 0 else 0
```

توضیح: ابتدا بازده روزانه S&P 500 را محاسبه و با index استراتژی هم‌راستا می‌کنیم (`reindex`)، سپس بازده فعال (`active_ret`) را با کم کردن بازده benchmark از بازده استراتژی به دست می‌آوریم. tracking error انحراف معیار این بازده فعال است که برای سالانه‌سازی در `sqrt(Day_Per_Year)` ضرب می‌شود. در نهایت میانگین بازده فعال را سالانه می‌کنیم (ضرب در `Day_Per_Year`) و بر tracking error تقسیم می‌کنیم.

درآخر معیار ها برای این استراتژی من به صورت زیر درآمد:

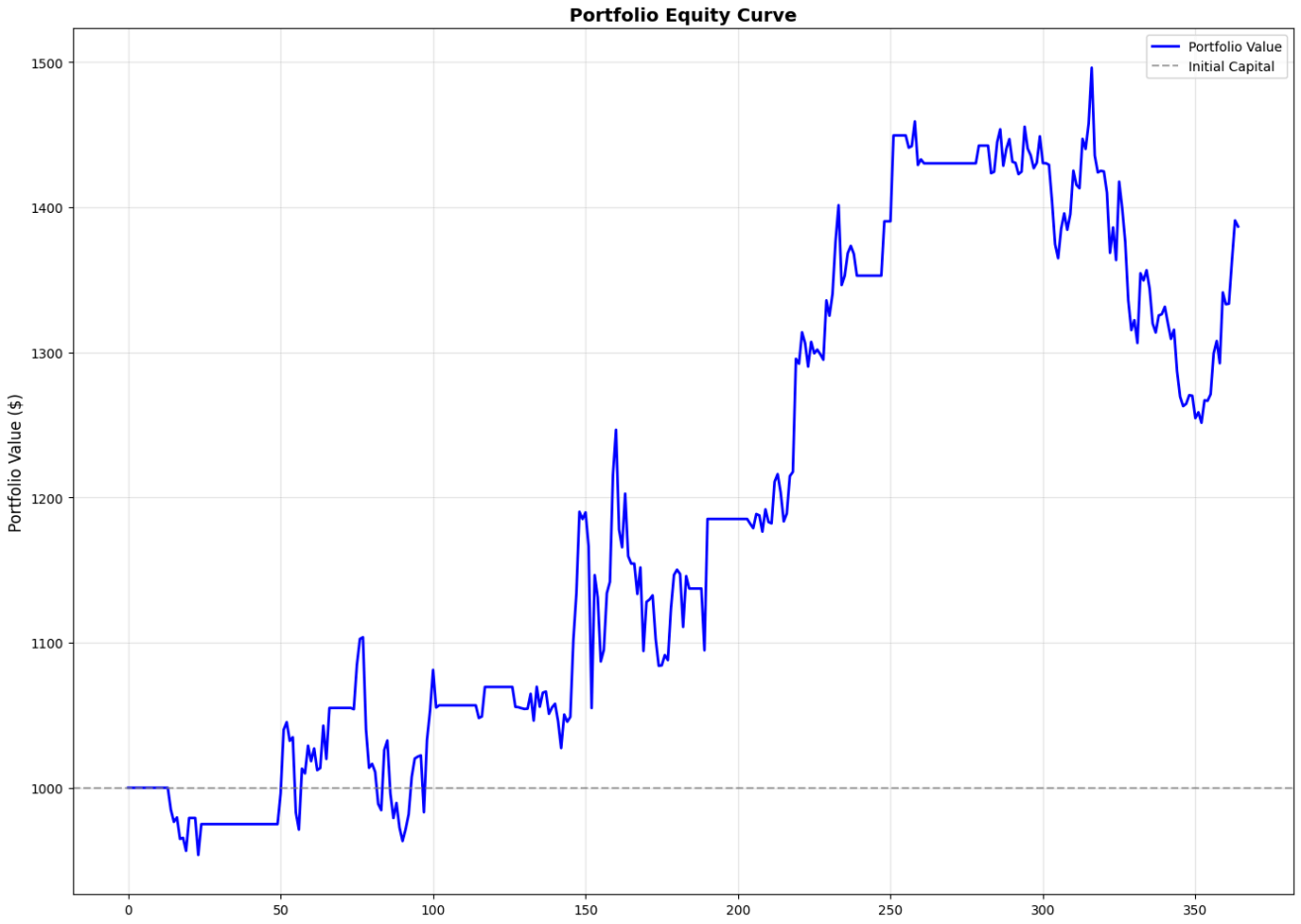
```
Metric                    | Value
----------------------------------------
Net Profit                | $386.59
Gross Profit              | $2,395.54
ROI                       | 38.66%
Annualized Return (CAGR)  | 38.66%
Annualized Volatility     | 34.15%
Max Drawdown              | -16.35%
Max DD Duration           | 69 Days
----------------------------------------
Sharpe Ratio              | 1.04
Sortino Ratio             | 0.93
Information Ratio         | 0.68
```

Visualization

در این بخش equity curve را پلات و نمایش می‌دهیم:

```
fig, (ax1) = plt.subplots(1, 1, figsize=(14, 10))

ax1.plot(df.index, df['equity'], label='Portfolio Value', linewidth=2,
color='blue')
ax1.axhline(y=initial_capital, color='gray', linestyle='--',
label='Initial Capital', alpha=0.7)
ax1.set_title('Portfolio Equity Curve', fontsize=14, fontweight='bold')
ax1.set_ylabel('Portfolio Value ($)', fontsize=12)
ax1.legend()
ax1.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

عکس:

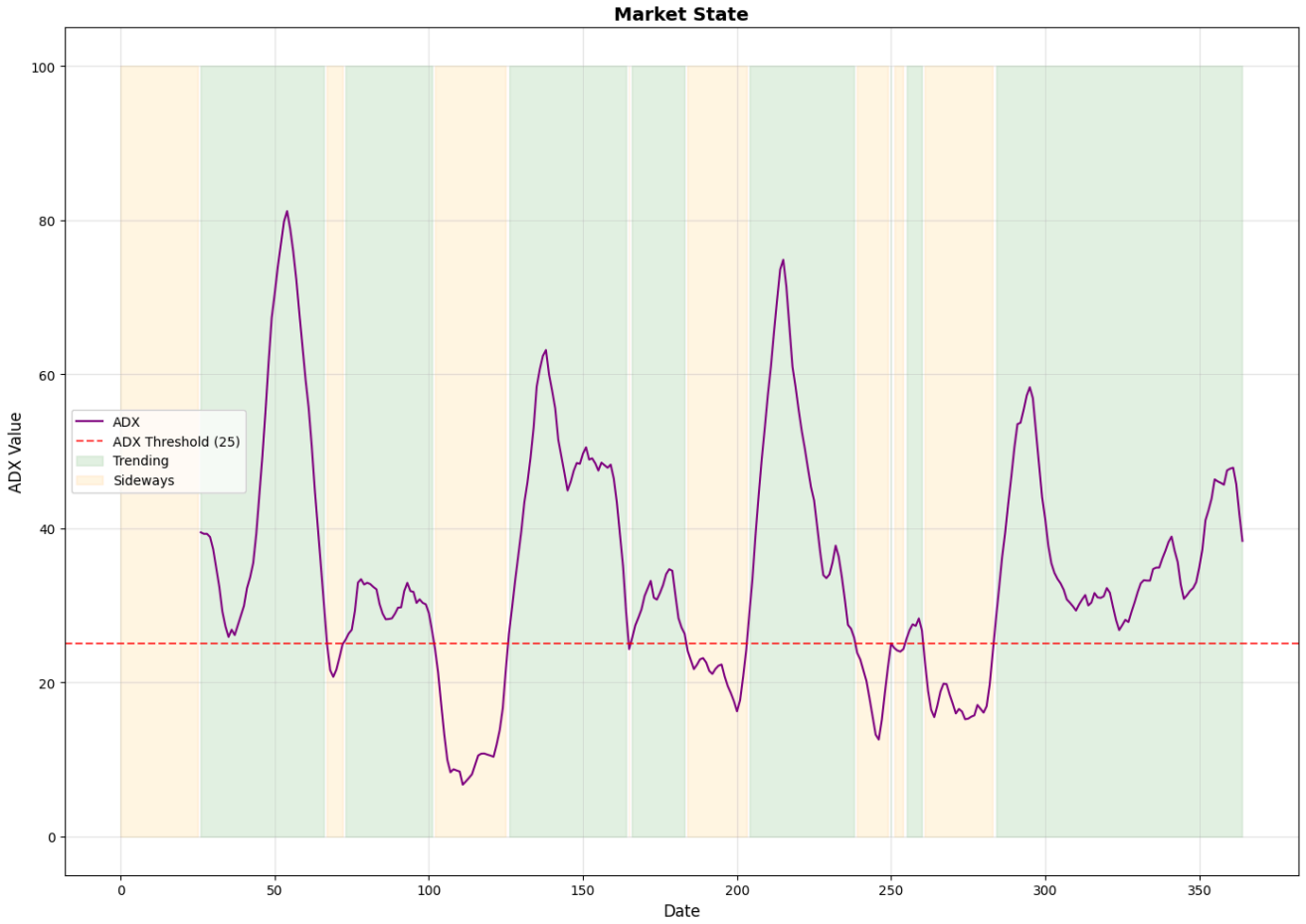


دراینجا هم من نمای کلی استیت بازار رو نشون دادم:

```
fig, (ax1) = plt.subplots(1, 1, figsize=(14, 10))

ax1.plot(df.index, df['ADX'], label='ADX', linewidth=1.5, color='purple')
ax1.axhline(y=25, color='red', linestyle='--', label='ADX Threshold (25)',
alpha=0.7)
ax1.fill_between(df.index, 0, 100, where=df['market_state']=='trending',
                    alpha=0.1, color='green', label='Trending')
```

```
ax1.fill_between(df.index, 0, 100, where=df['market_state']=='sideways',
                    alpha=0.1, color='orange', label='Sideways')
ax1.set_title('Market State', fontsize=14, fontweight='bold')
ax1.set_ylabel('ADX Value', fontsize=12)
ax1.set_xlabel('Date', fontsize=12)
ax1.legend()
ax1.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

عکس:

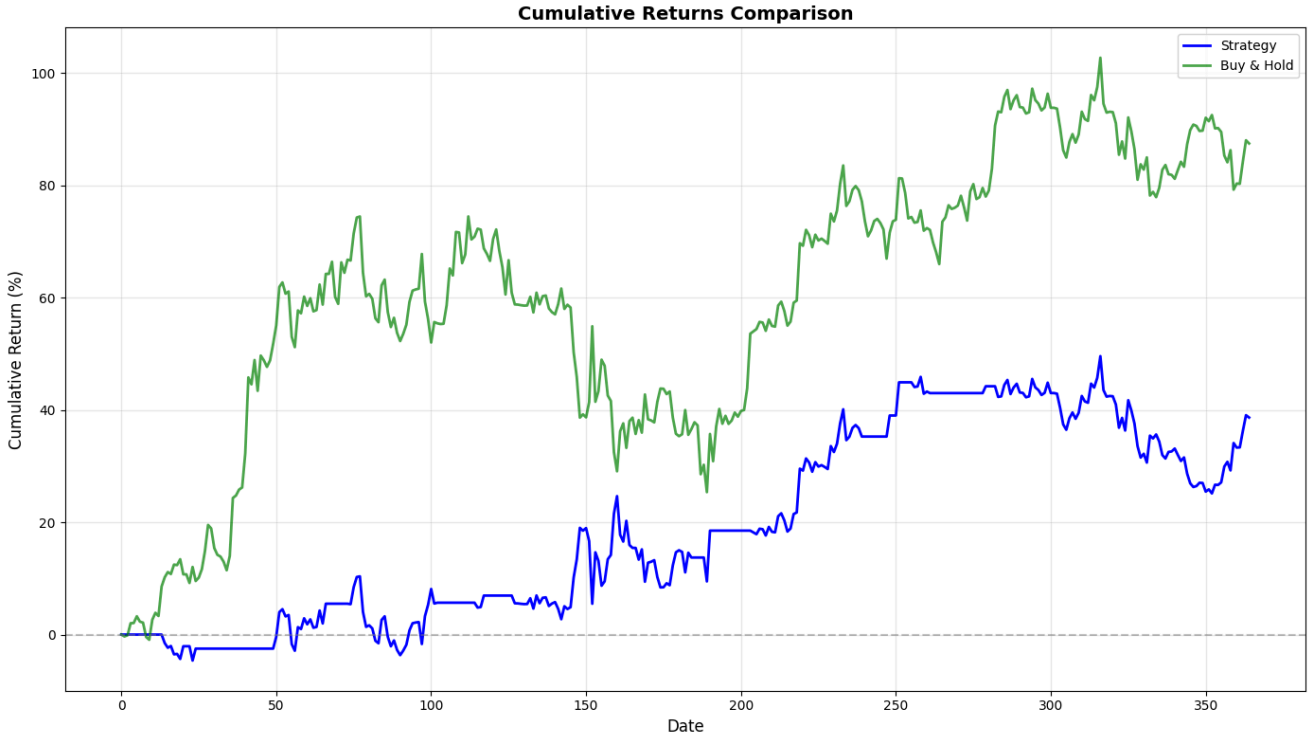


یکی دیگر از نمایش ها، نمایش مقایسه استراژی ما با buy and hold ساده بود:

```
fig, ax = plt.subplots(figsize=(14, 8))
ax.plot(df.index, (df['equity'] / initial_capital - 1) * 100,
label='Strategy', linewidth=2, color='blue')

buy_hold_returns = (df['Close'] / df['Close'].iloc[0] - 1) * 100
ax.plot(df.index, buy_hold_returns, label='Buy & Hold', linewidth=2,
color='green', alpha=0.7)

ax.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
ax.set_title('Cumulative Returns Comparison', fontsize=14,
fontweight='bold')
```

```
ax.set_ylabel('Cumulative Return (%)', fontsize=12)
ax.set_xlabel('Date', fontsize=12)
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

عکس:

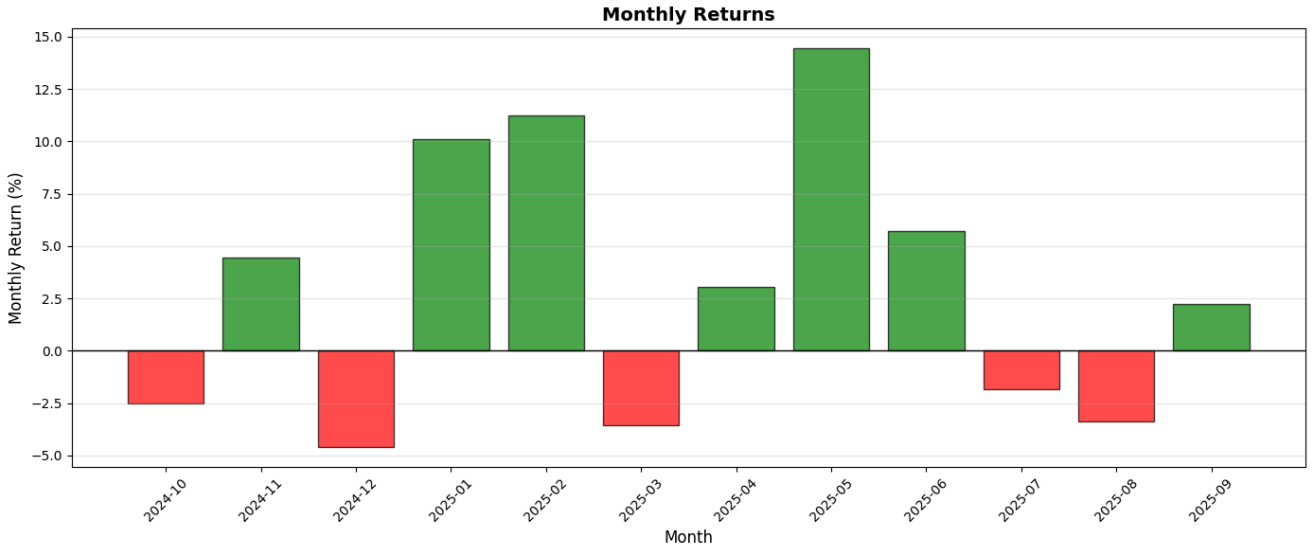


آخرین پلات هم پلات سوددهی هر ماه بود تا بهتر متوجه ماه هایی سودده و ضرررده بشویم:

```
monthly_returns = df.set_index('Date')
['strategy_returns'].resample('ME').apply(lambda x: (1 + x).prod() - 1)
monthly_returns_pct = monthly_returns * 100

if len(monthly_returns_pct) > 0:
    fig, ax = plt.subplots(figsize=(14, 6))

    colors = ['red' if x < 0 else 'green' for x in monthly_returns_pct]
    ax.bar(range(len(monthly_returns_pct)), monthly_returns_pct,
color=colors, alpha=0.7, edgecolor='black')
    ax.axhline(y=0, color='black', linewidth=1)
    ax.set_title('Monthly Returns', fontsize=14, fontweight='bold')
    ax.set_ylabel('Monthly Return (%)', fontsize=12)
    ax.set_xlabel('Month', fontsize=12)
    ax.set_xticks(range(len(monthly_returns_pct)))
    ax.set_xticklabels([d.strftime('%Y-%m') for d in
monthly_returns_pct.index], rotation=45)
```

```
    ax.grid(True, alpha=0.3, axis='y')

    plt.tight_layout()
    plt.show()
```

عکس:



Questions

1. پژوهشگر برای یک تست یک استراتژی شاخصی از لیست شرکتهای S&P 500 امروز استفاده میکند اما داده های قیمتی ۱۵ سال گذشته را برای همان شرکتها میگیرد بنابراین شرکتهایی که در گذشته حذف شده اند یا ورشکست شده اند در داده وجود ندارند.

Answer:

بله، در این سناریو بایاس بقا (Survivorship Bias) رخ داده است. به این دلیل که پژوهشگر برای بک‌تست یک استراتژی، از لیست فعلی شرکت‌های موفق S&P 500 استفاده می‌کند و داده‌های تاریخی ۱۵ ساله را تنها برای همین شرکت‌های بازمانده جمع‌آوری می‌نماید، در نتیجه، شرکت‌هایی که در طول این ۱۵ سال ورشکست شده، ادغام گشته یا به دلیل عملکرد ضعیف از لیست S&P 500 خارج شده‌اند، در مجموعه داده وجود ندارند. این فیلتر کردن باعث می‌شود که نتایج عملکرد استراتژی غیرواقعی و الکی بشود و بهتر از آنچه که در واقعیت و بازار live رخ می‌داده نشان داده بشود، زیرا ضرر و زیان‌های ناشی از سرمایه‌گذاری در شرکت‌های شکست‌خورده نادیده گرفته شده‌اند.

---

2. تحلیل گر تصمیم میگیرد داده هایی را که نوسان غیر عادی دارند مثلاً تغییر بیش از ۱۰ در روز حذف کند تا مدل تمیزتر آموزش ببیند این داده ها مربوط به زمانهایی هستند که بازار به شدت ناپایدار بوده است.

Answer:

بله، در این سناریو **بایاس انتخاب (Selection Bias)** و به‌طور خاص‌تر **بایاس حذف داده‌های پرت (Outlier Removal Bias)** رخ داده است. به این دلیل که تحلیل‌گر با حذف داده‌هایی که نوسان غیرعادی (بیش از ۱۰ درصد) دارند، در واقع **ریسک دنباله‌دار (Tail Risk)** و ناپایداری‌های شدید بازار را که بخش مهمی هستند را از مدل خود حذف کرده است.

این عمل باعث می‌شود مدل در یک محیط تمیزتر و غیر واقعی آموزش ببیند و در نتیجه، وقتی در شرایط واقعی بازار (شامل نوسانات شدید) استفاده شود، عملکرد آن غیرقابل اعتماد خواهد بود و ریسک‌های بازار را کم‌تر از حد واقعی حساب می‌کند.

---

3. در استراتژی انتخاب سهام تحلیل گر هر ماه سهامی را انتخاب میکند که بازدهشان بالاتر از میانگین S&P 500 در همان ماه بوده است در یک تست این میانگین را از داده های کل ماه محاسبه میکند.

Answer:

بله، در این سناریو **بایاس نگاه به آینده (Look-ahead Bias)** وجود دارد. به این دلیل که تحلیل‌گر در فرآیند بک‌تست خود، از **اطلاعاتی** (میانگین بازده S&P 500 در کل ماه جاری) استفاده می‌کند که در زمان **واقعی تصمیم‌گیری** (ابتدای ماه) در دسترس نبوده است. از آنجایی که بازده کامل ماه S&P 500 تنها در پایان آن ماه مشخص می‌شود، فرض استفاده از این میانگین برای انتخاب سهام در ابتدای ماه، به معنای **پیش‌بینی کامل و بی‌نقص** آینده است و در نتیجه، عملکرد استراتژی به‌طور غیرواقعی و گمراه‌کننده‌ا بهتر از حد واقعی می شود.